

Agile Project Management:

From Self-Managing Teams to Large-Scale Development

Tore Dybå
SINTEF
Trondheim, Norway
tored@sintef.no

Torgeir Dingsøy
SINTEF
Trondheim, Norway
torgeird@sintef.no

Abstract—Agile software development represents a new approach for planning and managing software projects. It puts less emphasis on up-front plans and strict control and relies more on informal collaboration, coordination, and learning.

This briefing provides a characterization and definition of agile project management based on extensive studies of large-scale industrial projects. It explains the circumstances behind the change from traditional management with its focus on direct supervision and standardization of work processes, to the newer, agile focus on self-managing teams, including its opportunities and benefits, but also its complexity and challenges. The main focus of the briefing is the four principles of agile project management: minimum critical specification, autonomous teams, redundancy, and feedback and learning.

The briefing is intended for researchers, practitioners and educators in software engineering, especially project managers. For researchers, an updated state of the art will be uncovered, and the presentation will be based on current best evidence. For practitioners, principles, processes, and key success factors will be outlined and a successful large-scale case study of agile project management will be presented. For educators, the briefing will provide the basis for developing course material.

Index Terms—Software Engineering, Large-Scale, Project Management, Agile Development, Self-Management, Portfolio Management

I. INTRODUCTION

A software project can be seen as a collection of activities that create an identifiable outcome of value. In its simplest form, project management consists of planning, executing, and monitoring these activities. However, the high costs and failure rates of software projects continue to engage researchers and practitioners, and despite several advances, the effective management of software projects is still a critical challenge.

Managing the unique and complex processes that constitute a project involves the implementation of specific management activities. In software development, as in most other businesses, there has been a tendency toward standardizing these activities by means of formalized, generic project management methodologies like, PRINCE2, which was developed and championed by the UK government. Although there is a global conception of the project management phenomenon, there is no unified theory of project management or well-defined measures of project success.

In large software projects, an adequate representation of all the technological, organizational, and environmental states that

might have a significant influence on the project's outcome of value, or of the causal relationships, is simply beyond the capabilities of the project team.

A further challenge is that the information needed to understand most software problems depends upon one's idea for solving them. The kind of problems that software projects deal with tend to be unique and difficult to formulate and solutions tend to evolve continually as developers gain a greater comprehension of what must be solved.

Adding to the complexity of the problem and its solution is the fast-changing and highly uncertain environment, for example, market turbulence and changes in customer requirements and project goals. It is necessary therefore to accept that our assumptions and predictions about future events will, by nature, be uncertain. When managing software projects, we need to be extremely cautious of extrapolating past trends or relying too heavily on past experience. Trends peter out, and the future is full of unexpected developments as well as unpredictable human behavior.

The greater the uncertainty inherent in a project, the more the project has to move from traditional approaches that are based on a fixed sequence of activities to approaches that allow to redefine the activities—or even the structure of the project plan—in midcourse. Therefore, as the project complexity and uncertainty increase, managers need to go beyond traditional risk management; adopting roles and techniques oriented less toward planning and more toward flexibility and learning.

II. AGILE PROJECT MANAGEMENT

At its core, this is what agile project management is about – it's about managing the impact of complexity and uncertainty on a project, recognizing

- The need for a dramatically shorter time frame between planning and execution.
- That planning an action does not provide all the details of its implementation.
- That creativity and learning are necessary to make sense of the environment.

Unlike the linear sequence of well-defined activities of traditional project management, agile project management is characterized by short cycles of iterative and incremental delivery of product features and continuous integration of code changes. Agile project management introduces changes in

management roles as well as in practices; it changes the nature of collaboration, coordination, and communication in software projects.

A fundamental property of software is its nonphysical form; software code is essentially a large set of abstract instructions possessing unlimited complexity, flexibility, and revisability. Software exhibits nonlinear behavior and does not conform to laws of nature. One consequence is that it is inherently hard to build models of software that allow accurate reasoning about the system's qualities. Agile project management addresses these basic properties of software and breaks away from the linear sequence of well-defined activities of traditional project management. It shifts focus from up-front planning to execution. In doing so, agile project management moves from traditional command and control structures to shared decision-making, self-management, and learning in software teams to deal with the complexity and unpredictability of the problem-solving activities of software projects.

III. PRINCIPLES OF AGILE PROJECT MANAGEMENT

Based on our extensive experience and studies of a multitude of agile projects during the last decade, the briefing offers the following set of socio-technical principles of agile project management:

- *Minimum critical specification*: This principle has two aspects; the first is that no more should be specified than is absolutely essential; the second requires that the team identify what is critical to overall success. This means that the system requirements should be precise about what has to be done, but not about how to do it, and that the use of rules, standards, and predefined procedures is kept to an absolute minimum. Focus should be on the larger system requirements and boundary conditions, leaving as many design decisions as possible to those closest to the work.
- *Autonomous teams*: This principle is based on the premise that autonomous, or self-managing, teams are a prerequisite for the success of innovative software projects. Such teams offer potential advantages over traditionally managed teams because they bring decision-making authority to the level of operational problems and uncertainties and thus increase the speed and accuracy of problem solving.
- *Redundancy*: This principle is concerned with the overlap in individuals' knowledge and skills in order to create common references for people's creation of new knowledge; as the level of redundancy increases within the team, individuals will find it easier to share new knowledge and the project will be able to coordinate its work more effectively. Therefore, this principle implies that each member of the team should be skilled in more than one function so that the project becomes more flexible and adaptive, which allows a function to be performed in many ways utilizing different people.

- *Feedback and learning*: The focus on project execution rather than on up-front planning in agile projects, leads to an intertwinement of learning and work, and of problem specification and solution. The complexity and unpredictability of software problems are typical of "wicked" problems, which are difficult to define until they are nearly solved. For such problems, requirements cannot be completely specified until most of the system is built and used. At the same time, the system cannot be built without specifying what is to be built. Furthermore, the problem is never really solved as improvements can always be made.

Together, these principles lay the foundation for successfully planning, executing, and monitoring the activities of a software project while allowing openness to define the details in each individual case according to the project's specific context.

IV. FROM SELF-MANAGING TEAMS TO LARGE-SCALE DEVELOPMENT

The briefing shows how agile project management was applied in the largest project with extensive use of agile principles so far in Norway; the recently completed Perform project for the Norwegian Public Service Pension Fund. This four-year project developed a system to administer pensions for 960,000 citizens. Thirteen Scrum teams developed 2,500 user stories, using about 800,000 person hours. The project is considered the largest agile success project in Norway so far.

The central leadership in the project consisted of a project director who mainly focused on external relations, a project manager focusing on the operations, and four sub-project managers responsible for business, development, architecture and test, respectively.

All teams were self-managing, but were configured with the same set of roles: scrum master, business analyst, team architect, senior developers, junior developers, and tester; 7-9 people in total for each team. These roles were fundamental in coordinating tasks and resolving dependencies across teams. Dependencies were also resolved in arenas, which included daily scrums and scrum of scrums.

The briefing also explains the overall development approach, which consisted of a product backlog process, a construction process with 5-7 sprints and an approval process; as well as the relationship between the project's master plan, release plans and sprint plans.

The briefing is based on the speakers' recent book chapter [1].

REFERENCES

- [1] T. Dybå, T. Dingsøy, N.B. Moe, "Agile Project Management," in G. Ruhe & C. Wohlin (Eds.) *Software Project Management in a Changing World*, Berlin: Springer Verlag, 2014, pp. 277-300. doi:10.1007/978-3-642-55035-5_11.